Prof. Anderson Domingues - anderson.domingues@pucrs.br

Trabalho T2 - 2026/1

Instruções:

- Entregar SOMENTE O LINK do seu repositório (Github, Gitlab, BitBucket, etc.) até o dia **20/11/2026** via Moodle. Seu repositório deverá permitir acesso público. No seu repositório, inclua os scripts e quaisquer arquivos necessários para a execução do projeto.

Objetivo: Este trabalho visa explorar o uso de Scan Chain em projetos de hardware.

Enunciado: O trabalho consiste na reprodução de um tutorial para a geração de padrões de teste utilizando as ferramentas Genus e Modus (Cadence), bem como a adaptação do projeto para o teste do módulo em anexo.

- **[0,5 ponto]** Execução do tutorial fornecido, utilizando as ferramentas Genus e Modus.
- **[1,5 ponto]** Adaptação do projeto para a geração de padrões de teste do módulo em anexo, utilizado no trabalho T1 (inserção de *scan registers*).
- **[1,5 ponto]** Geração e execução de padrões de teste, incluindo scripts de automação.
- **[4,0 pontos]** Experimentação com variação dos parâmetros do módulo em anexo. Apresentar a curva de inserção de registradores (área) em função dos parâmetros, bem como os relatórios de área e potência (default, Genus).
- **[1,0 ponto]** Apresentação do trabalho em aula, com participação da Ensilica. A apresentação acontecerá dia **24/11/2026**. **NOTA**: não é a data de entrega do trabalho.
- **[1,5 pontos]** Relatório apresentando resultados de exploração de espaço de teste (cobertura %) para diferentes parâmetros do módulo em anexo.

OBS.: Módulo em anexo abaixo:

```
//============================================================================
// stream_merge.sv  --  Reference (golden) implementation
//
// Two valid/ready PACKET streams merged into one output stream through an
// internal FIFO. Each input carries packets framed by sop/eop. A weighted
// round-robin arbiter on the push side chooses which input to serve, and it
// serves a whole packet before it is allowed to switch (packet atomicity).
//
// PUCRS - 98G10-4 Confiabilidade e Seguranca de Hardware
// Desafio 1 - EnSilica
//============================================================================

`timescale 1ns/1ps
```

```
module stream_merge #(
  parameter int DATA_W = 32,
  parameter int DEPTH  = 8,      // must be a power of two, >= 2
  parameter int WMAX   = 7       // maximum configurable weight (>= 1)
)(
  input  logic                          clk,
  input  logic                          rst_n,

  // configuration (must be stable while rst_n is low)
  input  logic [$clog2(DEPTH+1)-1:0]  cfg_af_thresh,
  input  logic [$clog2(WMAX+1)-1:0]   cfg_weight0,
  input  logic [$clog2(WMAX+1)-1:0]   cfg_weight1,

  // input stream 0 (packet framed by sop/eop)
  input  logic                          in0_valid,
  input  logic [DATA_W-1:0]             in0_data,
  input  logic                          in0_sop,
  input  logic                          in0_eop,
  output logic                          in0_ready,

  // input stream 1 (packet framed by sop/eop)
  input  logic                          in1_valid,
  input  logic [DATA_W-1:0]             in1_data,
  input  logic                          in1_sop,
  input  logic                          in1_eop,
  output logic                          in1_ready,

  // output stream
  output logic                          out_valid,
  output logic [DATA_W-1:0]             out_data,
  output logic                          out_src,
  output logic                          out_sop,
  output logic                          out_eop,
  input  logic                          out_ready,

  // status
  output logic [$clog2(DEPTH+1)-1:0]  occupancy,
  output logic                          empty,
  output logic                          full,
  output logic                          almost_full
);

  localparam int PTR_W = $clog2(DEPTH);
  localparam int CNT_W = $clog2(DEPTH+1);
  localparam int WT_W  = $clog2(WMAX+1);

  // FIFO storage: payload plus the tags that travel with each item.
  logic [DATA_W-1:0] mem_data [DEPTH];
  logic              mem_src  [DEPTH];
  logic              mem_sop  [DEPTH];
  logic              mem_eop  [DEPTH];
  logic [PTR_W-1:0]  wr_ptr, rd_ptr;
  logic [CNT_W-1:0]  occ;
```

```
  // Arbiter state -- NONE of this is visible at the ports. A scoreboard cannot
  // see it; only a spec-derived model can reproduce it.
  logic              locked;    // currently in the middle of a packet?
  logic              lock_src;  // ...from which input?
  logic [WT_W-1:0]   credit0;   // packets input 0 may still start this round
  logic [WT_W-1:0]   credit1;   // packets input 1 may still start this round

  //------------------------------------------------------------------------
  // Push-side arbitration
  //
  // Two nested rules:
  //   1. Packet atomicity. Once an item that is not eop has been accepted, the
  //      arbiter is LOCKED to that input and serves it exclusively until it
  //      accepts that input's eop -- even if the input bubbles mid-packet.
  //   2. Weighted round-robin at a packet boundary. When free, each input may
  //      START up to cfg_weight_i packets per round; starting one consumes a
  //      credit. When an offering input has no credit left, both credits
  //      recharge to their weights in the same cycle, so progress never stalls
  //      even if only one input is active.
  //------------------------------------------------------------------------
  logic off0, off1;              // an input is offering the start of a packet
  logic recharge;                // credits are refilled this cycle
  logic [WT_W-1:0] ec0, ec1;     // effective credits after any recharge
  logic elig0, elig1;            // eligible to start a packet this cycle
  logic serve;                   // input selected this cycle (0 or 1)
  logic serve_ok;                // an accept can actually happen this cycle
  logic push, pop;
  logic acc_sop, acc_eop;        // framing bits of the item being accepted
  assign off0 = in0_valid;
  assign off1 = in1_valid;

  // Recharge when free, at least one input offers, and no offering input still
  // holds credit. This single line is what keeps a lone active input flowing.
  assign recharge = !locked && (off0 || off1) && !((off0 && (credit0 != '0')) ||
(off1 && (credit1 != '0')));

  assign ec0 = recharge ? cfg_weight0 : credit0;
  assign ec1 = recharge ? cfg_weight1 : credit1;
  assign elig0 = off0 && (ec0 != '0);
  assign elig1 = off1 && (ec1 != '0);

  always_comb begin
    if (locked)              serve = lock_src;   // atomicity: finish the packet
    else if (elig0 && elig1) serve = 1'b0;       // both eligible: weighted
                                                 // burst favours input 0
    else if (elig1)          serve = 1'b1;
    else                     serve = 1'b0;
  end

  // Can an item actually be accepted this cycle? When locked, only that the
  // buffer has room. When free, the selected input must also be eligible.
  assign serve_ok = !full && (locked ? 1'b1 : (serve ? elig1 : elig0));
```

```
// REQ-RST-1: ready is forced low while in reset (no input can be accepted).
assign in0_ready = rst_n && serve_ok && (serve == 1'b0);
assign in1_ready = rst_n && serve_ok && (serve == 1'b1);

assign push = (in0_valid && in0_ready) || (in1_valid && in1_ready);
assign pop  = out_valid && out_ready;

assign acc_sop = serve ? in1_sop : in0_sop;
assign acc_eop = serve ? in1_eop : in0_eop;

//------------------------------------------------------------------------------
// Status flags
//------------------------------------------------------------------------------
assign full        = (occ == CNT_W'(DEPTH));
assign empty       = (occ == '0);
assign almost_full = (occ >= cfg_af_thresh);
assign occupancy   = rst_n ? occ : '0; // REQ-RST-1: reads zero while in reset

//------------------------------------------------------------------------------
// Output side (show-ahead: the head of the FIFO is presented combinationally)
//------------------------------------------------------------------------------
assign out_valid = rst_n && !empty; // REQ-RST-1: no valid output while reset
assign out_data  = mem_data[rd_ptr];
assign out_src   = mem_src[rd_ptr];
assign out_sop   = mem_sop[rd_ptr];
assign out_eop   = mem_eop[rd_ptr];

//------------------------------------------------------------------------------
// Storage, pointers and arbiter state
//------------------------------------------------------------------------------

always_ff @(posedge clk) begin
  if (!rst_n) begin
    wr_ptr   <= '0;
    rd_ptr   <= '0;
    occ      <= '0;
    locked   <= 1'b0;
    lock_src <= 1'b0;
    credit0  <= '0;        // first packet forces a recharge to the weights
    credit1  <= '0;
  end

  else begin
    if (push) begin
      mem_data[wr_ptr] <= serve ? in1_data : in0_data;
      mem_src [wr_ptr] <= serve;
      mem_sop [wr_ptr] <= acc_sop;
      mem_eop [wr_ptr] <= acc_eop;
      wr_ptr           <= wr_ptr + 1'b1;   // wraps naturally (DEPTH is 2^n)

      // Lock until this packet's eop is accepted.
      locked   <= ~acc_eop;
```

```
        lock_src <= serve;

        // Consume a credit only when a NEW packet starts (i.e. when free).
        if (!locked) begin
          credit0 <= (serve == 1'b0) ? (ec0 - 1'b1) : ec0;
          credit1 <= (serve == 1'b1) ? (ec1 - 1'b1) : ec1;
        end
      end

      if (pop) begin
        rd_ptr <= rd_ptr + 1'b1;
      end

      case ({push, pop})
        2'b10:   occ <= occ + 1'b1;
        2'b01:   occ <= occ - 1'b1;
        default: occ <= occ;                 // 00 idle, 11 push+pop cancel out
      endcase
    end
  end
endmodule
```